

Formal Verification of Onion Franking Security Properties Using Tamarin

Kayla Casey
*University of North
Carolina at Chapel Hill*
Chapel Hill, US
kaylaren@email.unc.edu

Sarah Haddix
*University of North
Carolina at Chapel Hill*
Chapel Hill, US
sarahbh@unc.edu

Bao-Nhi Vu
*University of North
Carolina at Chapel Hill*
Chapel Hill, US
baonhi@cs.unc.edu

Abstract—This paper presents a formal verification of the correctness, unforgeability, and accountability security properties of Onion Franking. Its hallmark is the use of the Tamarin Prover to symbolically model and analyze a simplified version of Onion Franking under adversary models. The benefit is the first machine-checked formal verification of our target security properties in Onion Franking, addressing a gap where formal verification had not previously existed and eliminating reliance on manual proofs or informal reasoning. We first validate our formal verification by symbolically modeling and verifying the One-Time Pad, an encryption scheme with well-established security guarantees, as our toy example. We then formalize Onion Franking in the Tamarin Prover, defining and proving the lemmas that capture our target security properties. We evaluate our formal verification by analyzing the satisfaction of lemmas, the accuracy of the symbolic model, and the abstraction of cryptographic primitives and system behavior assumptions. Our evaluation demonstrates that the correctness, unforgeability, and accountability security properties of Onion Franking hold, offering a first step toward verified privacy-preserving abuse mitigation in end-to-end encrypted messaging systems.

The code and proofs presented in this work are publicly available at: <https://github.com/kaylacasey/onion-franking-formal-verification>.

I. INTRODUCTION

The increasing adoption of end-to-end encrypted (E2EE) messaging systems has raised new questions about how to balance user privacy while providing moderation and abuse prevention. Onion Franking, a cryptographic protocol proposed by Gregoire, Pierce, and Eskandarian (2024) [1], enables verifiable abuse reports in E2EE metadata-hiding communication systems. This setting entails that clients can send messages to each other without the messaging platform seeing the contents of the messages or any associated metadata (e.g. timestamps, both sender and receiver identities). The Onion Franking protocol allows clients to report abusive messages without disclosing any message data about non-reported messages. The balance between user privacy and abuse prevention is breached: if anonymity is broken, users lose trust in the system; if abuse reporting fails, harmful actors can exploit the platform with impunity.

Mindful of these consequences, we turn to formal verification to provide additional assurance of critical security properties. Formal verification allows us to prove that a model of a cryptographic protocol satisfies certain security properties

under a precise threat model. Without a proven guarantee, even well-designed cryptographic protocols can harbor vulnerabilities that adversaries can exploit, such as bypassing the abuse reporting mechanism to send harmful messages. To address this gap, we leverage the Tamarin Prover, a symbolic model checking and analysis tool that symbolically models cryptographic protocols using rewriting rules and proves security properties expressed in first-order logic [2]. Formal verification allows us to prove that a symbolic model of a cryptographic protocol satisfies certain security properties under a precise threat model. In our analysis, we seek to formally model Onion Franking with the Tamarin Prover to verify the correctness, unforgeability, and accountability security properties.

Our approach begins with modeling the One-Time Pad (OTP) cipher to validate our methodology and gain fluency in Tamarin’s expressive language. We then model Onion Franking’s V.1 Construction and formulate lemmas expressing the security properties of correctness, unforgeability, and accountability. These security properties ensure that valid reports are always verifiable, that abuse reports cannot be forged, and that malicious actors cannot escape responsibility [1].

The main contribution of this paper is the formal verification of the critical security properties of correctness, unforgeability, and accountability for Onion Franking using the Tamarin Prover. Our work provides an introduction to the formal assurance that Onion Franking resists key attacks within its intended threat model, offering a robust framework for verifying abuse accountability in private messaging systems. Furthermore, this paper serves as a practical reference for cryptographic researchers and system designers by illustrating how to model and verify the security properties of complex cryptographic protocols, strengthening the rigor of cryptographic research and the trust users place in privacy-preserving communication platforms.

II. BACKGROUND ON THE FORMAL VERIFICATION FRAMEWORK AND CRYPTOGRAPHIC FOUNDATIONS

A. Tamarin Prover

The Tamarin Prover is a formal verification tool for the symbolic modeling and analysis of cryptographic security protocols [2]. A model that specifies the cryptographic protocol,

potential adversary capabilities, and intended security properties are taken in as input. Security protocols are represented using a multiset rewriting rule language that defines a labeled transition system [2], [8]. The state of the labeled transition system is represented using a multiset of facts, allowing the specification of adversary knowledge, cryptographic messages, fresh values, and the state of the security protocol. The default adversary is a Dolev-Yao adversary [2], [3], an adversarial framework with full control of the network to eavesdrop on, block, modify, and inject messages. We construct proofs with the Tamarin Prover with its automated mode that uses deduction, mathematical reasoning, and heuristics to guide the proof search [2]. If the tool terminates when running a proof search and the target security properties are satisfied, it returns a proof of correctness; if the target security properties are violated, it provides a counterexample representing an attack [2]. There are two verification modes in Tamarin Prover: satisfiability (verifies a security property is satisfied by an existing trace) and validity (verifies that a security property holds for all possible traces) [8].

B. One-Time Pad

The One-Time Pad (OTP) cipher [4] is an encryption scheme that provides perfect secrecy. In other words, the ciphertext reveals no information about the plaintext, ensuring that the adversary's knowledge remains independent of the encrypted message. The OTP assumes that the cipher key is randomized, equal to the message length, and used once [4]. The OTP encrypts a ciphertext by XOR-ing each bit of a plaintext message with the corresponding bit of the key [4]. Decryption requires the recipient to know the key and apply XOR to the ciphertext [4]. Although impractical for large-scale use due to its strict requirements, the simplicity and security properties of the OTP make it an effective toy example.

C. End-to-End Encryption

End-to-end encryption (E2EE) is a security feature for secure communication that protects the confidentiality of message content from messaging systems [1]. Messages are encrypted on the sender's device and decrypted on the receiver's device. The messaging system, intermediate server, or third party has no access to the plaintext. This ensures that only the intended receiver can read message contents even if the messaging platform infrastructure is compromised [1]. E2EE is becoming standard in messaging systems, with the additive protection of metadata hiding being a future goal of research [1].

D. Onion Franking

Onion Franking is a cryptographic protocol designed to support abuse reporting in privacy-preserving messaging systems that use onion routing [1]. Onion routing is a mechanism of layering encryptions to hide sender and receiver identities as a message passes through a system of servers. Onion Franking introduces a mechanism for a sender to include a verifiable, hidden message authentication tag (MAC tag)

within the message. If a recipient wishes to report abuse, they can extract this tag and present it to a moderator, who can verify its authenticity without learning the sender's identity. The system ensures that only the moderator can perform this verification, preserving deniability, and that no false reports can be constructed, preserving unforgeability. Onion Franking builds on cryptographic primitives such as commitments, signatures, and public key encryption, combining them in a novel way to enforce accountability without compromising user privacy.

E. Cryptographic Primitives in Onion Franking

Onion Franking leverages the following cryptographic primitives [1]:

- *Public-Key Encryption (PKE)*: Allows users to encrypt messages using a recipient's public key to form a ciphertext that can only be decrypted by the corresponding secret key of the receiver.
- *Digital Signatures*: Provides the authenticity of users and the integrity of information. A sender will sign a message with their private key, which can be verified by anyone holding the sender's public key. This ensures that a message has not been altered and that recipients can verify senders.
- *Commitment Schemes*: Enable a user to commit to a value while keeping it hidden, with the ability to reveal it later. A commitment can be both hiding, meaning that it reveals no information, and binding, meaning that it cannot be changed after committing.
- *Hash Functions (modeled as Random Oracles)*: Provide deterministic mappings from arbitrary-length inputs to fixed-size outputs. The hash function is treated as a black box that outputs random values for new inputs.

These cryptographic primitives are often used as the building blocks for other complex cryptographic protocols and are verified for the security properties they provide. The Tamarin Prover allows us to reason symbolically about how these primitives interact, which we will use to compose Onion Franking's V.1 Construction.

F. Security Properties of Onion Franking

Gregoire et al. [1] define the following security properties within Onion Franking:

- *Correctness*: If an honest sender generates a franking tag and an honest recipient reports it correctly, then the moderator should accept the report.
- *Unforgeability*: A malicious client should not be able to forge an abuse report, even when colluding with malicious servers. This ensures that no user can falsely blame another for a message they did not send.
- *Accountability*: If a message is sent through the messaging system, it should be possible to verify that it was sent, even if the sender deviates from the protocol. This prevents malicious clients from escaping responsibility.
- *Deniability*: No third party, including compromised servers, should be able to verify a message or report;

only the moderator can do so. This maintains the privacy and deniability of communication outside of the abuse reporting context.

- *Confidentiality*: The integration of Onion Franking must not compromise the confidentiality or metadata-hiding guarantees of the underlying messaging system, except in the case of a reported message.

These properties are vital to Onion Franking’s goal of abuse reporting while preserving the core privacy protections of metadata-hiding E2EE messaging systems. It is important to note that the exact definition of security properties will vary across different cryptographic protocols.

III. THREAT MODEL ASSUMPTIONS

The threat model in our formal verification parallelizes the threat model presented by Gregoire et al. [1] and the Dolev-Yao adversary in the Tamarin Prover [2]. Onion Franking is designed to defend against adversaries who may act as arbitrary malicious clients [1]. The Tamarin Prover assumes an adversary with complete control over the communication channel but is unable to break cryptographic primitives (e.g. cannot guess private keys or invert hash functions without explicit modeling) [2]. Combining these two adversarial assumptions, our adversary is a malicious client who is capable of sending abusive messages, attempting to generate false reports, and can intercept, modify, delay, and replay network communication (i.e. has full control of the network) [1], [2]. Servers are assumed to remain available to process messages, even if the messages are malicious.

We do not consider global passive adversaries (i.e. full surveillance), side-channel attacks, or cryptographic breaks. These assumptions lie outside the scope of the Onion Franking protocol [1] and the Tamarin Prover threat model [2]. The verification goal is to ensure that the cryptographic protocol functions as expected against realistic threats, making private messaging secure and accountable. Verification of the deniability and confidentiality security properties assumes that an adversary colludes with servers within the messaging system. However, these security properties are a stretch goal and are not verified in this work.

IV. MODEL DESIGNS OF PROTOCOLS

A. Design of the One-Time Pad Model

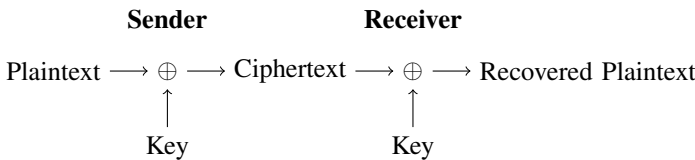


Fig. 1. OTP encryption and decryption flow. Sender XORs message and key to create ciphertext. Receiver XORs ciphertext and key to recover the message.

We began by modeling the OTP as a toy example to gain familiarity with Tamarin Prover’s symbolic modeling and

proof system. Our OTP design centers around three key rules: key generation, message sending, and message receiving. We also defined rules to describe how information can be leaked to the adversary in the protocol.

Key generation is modeled with the `GenerateKeyPair` rule, which simultaneously produces a fresh key and a fresh identifier (id) associated with a sender-receiver pair. Sending a message with the `Send` rule consumes the sender’s key share and outputs a ciphertext formed by XOR-ing the message and the key. Receiving a message with the `Receive` rule consumes the receiver’s key share and reconstructs the original plaintext by XOR-ing the ciphertext and the key. To model key freshness and prevent reuse, we associate every key with an identifier, ensuring one-time usage. A diagram illustrating the OTP model is shown in Figure 1.

B. Design of the Onion Franking Model

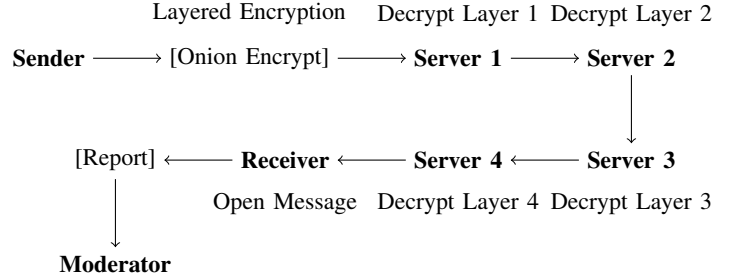


Fig. 2. Onion Franking flow modeled in Tamarin Prover: sender onion-encrypts the message, servers sequentially peel layers, receiver recovers the message and generates an abuse report, moderator verifies the report.

We next modeled the Onion Franking cryptographic protocol, closely following the V.1 Construction described by Gregoire et al. [1], adapted into Tamarin syntax. The protocol logic is divided across several key rules. The `GenerateSymmetricKey` rule establishes symmetric keys shared between a sender and receiver, while `GenerateOnionServerNetwork` generates fresh public/private key pairs for four onion servers and initializes a network identifier, ensuring consistent routing. To prevent confusion between different network setups, we enforce a `SingleNetwork` restriction, ensuring that only one onion server configuration is active at any given time.

Sending a message is modeled by the `Send` rule, which captures onion encryption across multiple servers, generation of a commitment tag using `Com_Commit`, signing of metadata by the moderator key using `Mac_Sign`, and layered masking of metadata using XOR operations. Each server peels one encryption layer and updates the metadata by removing one masking layer. This is modeled by simulating multiple decryption steps (using `adec` within the `Send` rule itself to reflect server behavior).

The `Read_and_Report` rule models how a receiver processes an incoming message: decrypting it with the shared symmetric key (using `sdec`), recovering the original franking

commitment, signature, and metadata by removing all masking layers, and constructing a report that will later be verified by the moderator. We represent the verification performed by the moderator through the `Moderate` rule, where the moderator checks both the validity of the franking commitment (using `Com_Open`) and the correctness of the MAC signature (using `Mac_Verify`).

In addition to the core protocol rules, we formalized restrictions and properties crucial for security proofs. The `TrustedReceive` restriction enforces that every message received must have originated from a valid message sent earlier, preventing forged messages. We then formalized key security properties such as *correctness* (honest reports are accepted), *unforgeability* (malicious reports are rejected), and *accountability* (misbehavior can be proven). Each security goal is encoded as a lemma using Tamarin Prover’s first-order logic syntax. A diagram illustrating the high-level flow of the Onion Franking model is provided in Figure 2.

V. IMPLEMENTATION

A. Implementation of the One-Time Pad Model

We implemented the OTP design by translating each conceptual design step into Tamarin Prover rules. Key generation, encryption, and decryption were modeled explicitly using `xor` built-ins. A small code snippet of the key generation rule can be seen below as an example to the reader of how we transitioned from the conceptual model to the Tamarin language.

```
rule GenerateKeyPair:
  [ Fr(~k), Fr(~id) ]
  --> [ KeyPairOne($S, $R, ~k, ~id),
        KeyPairTwo($S, $R, ~k, ~id) ]
```

In this example, the `GenerateKeyPair` rule creates a fresh key and id, and outputs two facts corresponding to shares of the key for the sender and receiver.

An important modeling choice we made was associating each key with a unique identifier (id) to ensure one-time key usage and to allow fine-grained reasoning in the security lemma. Without this id, counterexamples were generated where key reuse incorrectly satisfied the properties. The counterexample generated by Tamarin can be seen in Figure 3.

We then formulated and proved the `Secure` lemma, which states that if a message is sent and correctly received, no instance of the key should remain accessible to an adversary.

B. Implementation of the Onion Franking Model

The Onion Franking implementation was more complex due to the multistage encryption, metadata processing, and abuse reporting steps.

Each major function of the cryptographic protocol (e.g., onion encryption, onion peeling, metadata processing, moderation) was mapped to one or more Tamarin Prover rules. Complex cryptographic primitives like commitments and MACs were abstracted using equations, such as:

```
equations: Com_Open(Com_Commit(r,m), m, r) = true
```

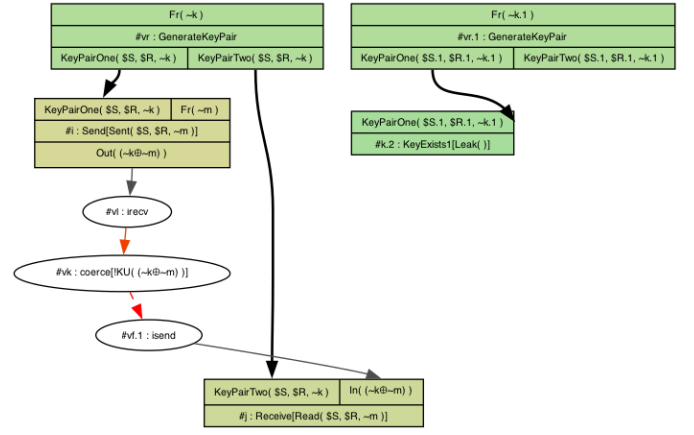


Fig. 3. Counterexample generated by Tamarin for initial OTP implementation without the id functionality added.

```
equations: Mac_Verify(k, m, Mac_Sign(k, m)) = true
```

The Onion Franking model defines three main rules: `Send` for encrypting and transmitting messages through layered onion encryption and masked metadata; `Read_and_Report` for a receiver to decrypt, unmask, and prepare an abuse report; and `Moderate` for the moderator to verify the validity of a reported message by checking the franking commitment and MAC signature.

The security properties of correctness, unforgeability, and accountability were specified as Tamarin lemmas. As an example, the `Unforgeability` property was encoded as:

```
lemma Unforgeability:
  "All m ctx #j.
  (Moderator_COM_OPEN(m, ctx, true)@#j &
   Moderator_MAC_Verify(m, ctx, true)@#j)
  ==>
  (Ex Sender Receiver #x.
   MessageSent(Sender, Receiver, m, ctx)@#x &
   #x < #j) "
```

We encountered challenges during implementation, particularly regarding XOR handling. While Onion Franking relies on repeated XOR-masking at each server, symbolic XOR in Tamarin Prover exhibits associative and commutative properties, leading to complex proof obligations. To mitigate this, we compressed the masking process into a single XOR using a derived random value, which reduced proof complexity without sacrificing the correctness of the protocol’s symbolic model. In addition, we attempted to utilize source lemmas aimed at simplifying the proof construction by relating outputs back to honest origins, but they proved to be unhelpful due to their difficulty in satisfying Tamarin Prover’s matching constraints and were ultimately commented out. Despite these challenges, we successfully modeled Onion Franking’s essential security behaviors within the Tamarin Prover framework.

VI. EVALUATION

For the One-Time Pad, we successfully verified the `Secure` lemma, proving that a message sent and received under OTP

guarantees that the associated key no longer leaks to the adversary. Early attempts without an id field led to a counterexample where a key was reused across multiple messages, demonstrating the importance of explicitly modeling key freshness. Adding the id field and associating keys to specific message sends resolved this issue and yielded a successful proof.

We ran the OTP implementation in Tamarin Prover’s interactive mode, using the `by Sorry` and `a (autoprove all)` commands to automatically initiate proof steps and confirm the secrecy guarantees of the One-Time Pad. The full proof is found in our GitHub repository.

For Onion Franking, we were able to partially prove the `Unforgeability` lemma under our threat model. While full verification was limited by Tamarin’s scalability challenges, such as path explosion, high memory usage, and ambiguity from symbolic XOR handling, the modeling effort itself provided significant value. In particular, during formal modeling, we found a minor inconsistency surfaced in the original Onion Franking V.1 Construction: while the messages sent through the network were going through servers in ascending order, they were being encrypted as if they were to be sent through the network in descending order. To fix this inaccuracy, we propose that the `Send` function change its last three lines to:

Algorithm 1 Corrected Fragment of `Send` Function

```

:
Initialize  $c_{3,0} \leftarrow \epsilon$ 
for  $i = 1$  to  $N$  do
   $c_{3,i} \leftarrow \text{Enc}(\mathbf{pk}_i, (c_{3,i-1}, r_i))$ 
end for
 $c_3 \leftarrow c_{3,N}$ 
Return  $c_1, c_2, c_3$ 

```

We include a proof of this indexing issue in the Appendix, along with a proposed correction to the protocol specification.

We faced roadblocks with Tamarin Prover exhausting computational resources or being terminated by the operating system, despite utilizing a machine with 96 GB of RAM. In addition to these scalability challenges, the lack of detailed debugging support and descriptive error messages in Tamarin Prover made it difficult to isolate the root causes of proof failures. Nevertheless, we believe there is significant value in modeling cryptographic security properties for formal verification, even when only partial proofs are achieved, as this process can still reveal subtle edge cases and unexpected vulnerabilities in protocol designs.

Beyond resource limitations, we observed that proving liveness properties was difficult. Tamarin Prover’s symbolic reasoning engine is better suited for proving reachability-style security properties, such as secrecy and authenticity, rather than full temporal guarantees expressed in CTL-like logic. As a workaround, we modeled an *executability* lemma, which proves the existence of at least one valid trace through

the protocol, instead of attempting to verify general liveness properties across all traces.

Finally, we note that Tamarin is particularly well-suited for protocols where the primary adversarial threat comes from the network, as in the Dolev-Yao model, over verifying low-level cryptographic details or nuanced adversarial models. Since Onion Franking involves a unique cryptographic structure and threat model in addition to network adversaries, modeling and verifying the full system proved especially challenging within Tamarin.

Despite the difficulties, the verification of `Unforgeability` in Onion Franking is a promising first step toward formally verified abuse reporting in private messaging systems, and more broadly, the practice of formally verifying cryptographic protocols. Future work relating to this project could involve exploring how to more efficiently translate protocols into Tamarin in an attempt to cut down on path explosion, fully verifying some of the security properties mentioned, and modeling the other variants of the Onion Franking protocol.

VII. STATE OF THE ART

This section discusses prior work on formally verifying cryptographic schemes and security protocols using the Tamarin Prover or other formal verification methods.

A. Symbolically Analyzing Security Protocols with Tamarin Prover

Basin et al. [5] describe Tamarin Prover’s core strengths, including its constraint-solving engine, rule-based multiset rewriting semantics, and ability to verify trace properties such as secrecy, authenticity, and resistance to replay attacks. Additionally, Tamarin Prover was successfully applied to security protocols within their intended adversary models, such as key exchange protocols, protocols with loops and mutable global state, and large-scale public key infrastructures [5]. Within these security protocols, both verifications and attacks were found. For example, the KAS1 key exchange protocol for messaging systems was verified for Key Independence and Key Compromise Impersonation [5]. However, Tamarin Prover found an attack on Key Independence for the STS-MAC security protocol for messaging systems when the adversary registers arbitrary public keys [5].

B. Symbolic Verification of Cryptographic Protocols

Basin et al. [6] used Tamarin Prover to verify complex real-world cryptographic protocols and verification problems beyond academia and into industrial use. Tamarin Prover was capable of finding errors, correcting designs, and supporting the development and standardization of cryptographic protocols. In TLS 1.3, an internet security protocol, Tamarin Prover found an attack on the delayed authentication mechanism [6]. Then, in 5G-AKA, a 5G key agreement protocol, Tamarin Prover discovered a privacy problem within the resynchronization protocol prior to deployment and ensured the authentication and secrecy security properties [6].

C. Formal Verification of the Signal Messaging Protocol

Cohn-Gordan et al. [7] present a formal verification of the Signal messaging protocol, a cryptographic messaging protocol adopted in messaging systems such as WhatsApp, Wire, and Facebook Messenger. Rather than using an automated verification tool, a manual formal security analysis was conducted by defining a new security model tailored for multi-stage authenticated key exchange protocols. The outcome of the paper was providing the first formal security guarantees for Signal by proving security within their defined model [7].

VIII. CONCLUSION

This paper presents the first symbolic model of the correctness, unforgeability, and accountability security properties for the Onion Franking protocol using the Tamarin Prover. We successfully modeled these security notions and partially proved the unforgeability property under our threat model. Although full verification was limited by scalability and resource constraints, our work demonstrates the feasibility and value of using formal methods to analyze complex, privacy-preserving messaging protocols. Additionally, we validated our modeling approach by formally verifying the security of the One-Time Pad as a toy example. By uncovering subtle issues, modeling key cryptographic behaviors, and partially proving critical properties, our work highlights the practical benefits of symbolic formal verification in strengthening the security foundations of cryptographic systems. Future work can build on this foundation to refine proofs, optimize symbolic models, and extend verification to broader constructions of Onion Franking.

IX. ACKNOWLEDGMENT

We acknowledge the use of generative AI tools for assistance with grammar and syntax refinement of this paper and our code. We also utilized AI-based support to better understand portions of the Tamarin Prover's documentation and to debug aspects of the model and proof development process.

REFERENCES

- [1] M. Gregoire, M. Pierce, and S. Eskandarian, "Onion franking: abuse reports for mix-based private messaging," Cryptology ePrint Archive, Paper 2024/1965, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1965.pdf>
- [2] The Tamarin Team, Tamarin Prover Manual: Security Protocol Analysis in the Symbolic Model, Oct. 30, 2024. [Online]. Available: <https://tamarin-prover.com>
- [3] Cervesato, "The Dolev-Yao intruder is the most powerful attacker," ITT Industries, Inc., Alexandria, VA, USA. [Online]. Available: <https://www.csl.sri.com/users/iliano/>
- [4] National Security Agency (NSA), "Cryptographic System," Declassified Friedman Documents. [Online]. Available: <https://www.nsa.gov/portals/75/documents/news-features/declassified-documents/friedman-documents/patent-equipment/folder-545/41781159082021.pdf>
- [5] D. Basin, C. Cremers, J. Dreier, and R. Sasse, "Symbolically Analyzing Security Protocols using TAMARIN," ACM SIGLOG News, vol. 4, no. 3, pp. 32-41, Oct. 2017. [Online]. Available: <https://ethz.ch/content/dam/ethz/special-interest/infk/inst-infsec/information-security-group-dam/research/publications/pub2017/siglog-tamarin.pdf>

- [6] D. Basin, C. Cremers, J. Dreier, R. Sasse, Tamarin: Verification of Large-Scale, Real World, Cryptographic Protocols. IEEE Security and Privacy Magazine, In press, (10.1109/msec.2022.3154689). (hal-03586826)
- [7] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, "A formal security analysis of the Signal messaging protocol," in 2017 IEEE European Symposium on Security and Privacy (EuroS&P), Paris, France, 2017, pp. 451-466, doi: 10.1109/EuroSP.2017.36.
- [8] F. Linker, Formal Verification of the Privacy Guarantees of Metadata-Hiding Communication Systems, M.S. thesis, Dept. of Computer Science, ETH Zürich, Zürich, Switzerland, 2022.

X. WHO DID WHAT

Kayla contributed to the paper writing, code documentation, cleaning and finalizing the GitHub repository, and assisting in modeling the OTP and lemmas in the Onion Franking implementation. Sarah did the majority of the code writing, assisted in cleaning up the paper, documenting the code, finding a machine with greater RAM to use, and discovering and proving the incorrect indexing finding in the Onion Franking protocol. Bao-Nhi contributed to writing the paper, checking for inconsistencies, and composing the paper and documentation.

XI. APPENDIX

Proof of Incorrect Indexing in Onion Franking for Construction V.1: We consider c_3 , as created by the loop in the Send function [1]. As written in the paper, $c_{3,i} = \text{Enc}(pk_i, (c_{3,i-1}, r_i))$ and makes up st_{i-1} . We know this because the Process function of each server takes st_{i-1} as an input.

Then, $\text{Dec}(ski_i, c_{3,i}) = (c_{3,i-1}, r_i)$ in S'_i 's Process function ($c_{3,i-1} = c'_3$, to match notation exactly). Then, we get $st_i = (c'_3, mrt') = (c_{3,i-1}, mrt')$ (mrt may be ignored) and return st_i on the last two lines of Server i 's Process function.

Now, let's consider $c_{3,i-1}$. By definition, $c_{3,i-1} = \text{Enc}(pk_{i-1}, (c_{3,i-2}, r_{i-1}))$. By the same reasoning applied before, $c_{3,i-1}$ makes up st_{i-2} .

Then, we have that $c_{3,i-1}$ makes up both st_{i-2} and st_i , which is a contradiction.

A more intuitive way to see this indexing error is that since we're sending something through the servers in the order $S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_N$, we should encrypt things in the manor $c_3 = \text{Enc}_1(\text{Enc}_2(\dots(\text{Enc}_N(z))))$, where $\text{Enc}_i(x) = \text{Enc}(pk_i, x)$ and z is the information we mean to send through the servers. All this proof means to claim is that the servers and the encryption have been indexed in contradictory ways.